Agentic AI in a Research Context

Contents

Part 1: The Agentic AI Proposition

Agentic AI is treated here as an efficiency tool: Both in terms of tasks we know and in terms of opportunities and directions that may be "out of sight" for now. The "Research Context" of the title is informed by real programs at the University of Washington. Before elaborating some case studies I want to emphasize common themes:

- Challenging: We choose research problems that are *complex* (but–we hope–tractable).
- Worthwhile: We regard the work as *important*, worth our time and effort.
- Communal: We benefit from dissemination and sharing of information and ideas with colleagues.

Case studies

Here are some brief descriptions of research programs at the University of Washington. In common they involve data science and use of the public cloud (AWS, Azure etcetera) for research infrastructure.

Bill Noble (Genomics) The Noble lab (Department of Genome Sciences) is developing Puget, a computational model that predicts gene expression across cell types by jointly integrating DNA sequence information with 3D chromatin organization data derived from Hi-C contact maps. Gene expression determines cell identity and developmental programs, and disrupted expression contributes to diseases including neurological disorders and cancers. Existing sequence-to-expression models like Enformer and Borzoi use DNA sequence alone and cannot represent cell type-specific chromatin contacts.

> A common computing workflow: Training a model (a neural network) for analysis specific to a particular type of data.

Paul Atkins (Asian Languages & Literature) Professor Atkins' group is developing a machine-learning system to extract ortho-

graphic data — specifically "jibo" (character matrices) — from premodern Japanese manuscripts written in cursive hiragana. Existing OCR tools for classical Japanese (like the Miwo app) skip the jibo step entirely, producing readings without capturing the underlying character-matrix information. This project builds a computer vision pipeline (Cascade R-CNN and MaskOCR architectures) trained on over 1 million labeled characters from the CODH kuzushiji dataset to recognize jibo at an intermediate step, then deploys the model as a web tool for scholars worldwide.

> Methods developed in computer science (OCR is Optical Character Recognition) are adopted to Digital Humanities (another neural network).

Jarod Alper (Mathematics) Professor Alper runs the UW Math AI Lab which hosts 11 research projects at the intersection of mathematics and AI. Compute-intensive projects include: training reinforcement learning agents (AlphaZero-style) to discover efficient arithmetic circuits for polynomials, building graph neural networks to classify knots from mosaic diagrams, fine-tuning language models to correct Lean proof assistant errors, developing theorem search tools using text embeddings, benchmarking LLMs on Lean code generation, and training AI agents for quantum error-correction circuit synthesis.

> Can AI systems advance progress in mathematics? Here generative AI is itself a potential means of finding proofs.

Kelly Michaelsen (UW Medicine, Anesthesiology) Professor Michaelsen's group is developing a computer vision system using smart eyewear to automate drug administration documentation in operating rooms to reduce medication errors. The system uses YOLO for syringe/vial detection and CLIP for drug identification (already published in Nature Medicine). The current phase focuses on determining syringe volume from video — framing it as a classification problem over video frames rather than requiring clean single-image segmentation. If fine-tuning standard models doesn't achieve the required accuracy, a custom backbone may be pretrained using the DINOv2 self-supervised framework on hundreds of hours of unlabeled OR footage.

> Clinical research: Improving patient care; working from existing specialized models such as YOLO.

Lucy Lu Wang (Information School) Professor Wang is investigating whether LLM abstention — a model's ability to refuse an-

swering when it lacks confidence, sufficient information, or risks producing harmful output — can be learned as a generalizable meta-capability that transfers across tasks and domains. Current mitigation strategies for hallucination and unsafe outputs tend to be task-specific; this project asks whether abstention behavior trained via LoRA adaptors on refusal-aware datasets can generalize to unseen tasks. The work extends to vision-language models (VLMs), creating a benchmark dataset for multimodal abstention scenarios such as ambiguous images or incomplete visual context.

Direct focus on the behavior of Language Models.

Pivot to the use of Agentic AI

The case studies provided above are intended to give a sense of *research context* where an Agentic AI might come in to play. To pivot to the rather deep topic of "How to…" I told the **Agentic AI** that I work with to construct three tips on how to use it when doing research. To make the ensuing remarks clear: The Agentic AI system I use goes by the brand name `kiro`. It was developed and released by Amazon Web Services as an adaptation of the VS Code IDE.

Here are the tips:

- Use specs to decompose your research question before writing code.
- Steer the AI with your domain knowledge.
- Treat the conversation as an iterative lab notebook.

I also checked in on kiro's familiarity with popular culture: 'Complete the phrase: "Never cross wits with a Sicilian…" ' Without missing a beat `kiro` responded "…when *death* is on the line."

Agentic AI appears to be a useful tool with a capacity for both direct tactical response and introspective, strategic behavior. (To acknowledge the philosophical situation for a moment: I use terms like *behavior* as shorthand for a *trick* or *simulacrum* or *apparent fluency* that emerges from some weighted arithmetic.) The underlying language model is built on a deep index of our collective knowledge base; so there is some good potential here. There are also two potential down sides I want to emphasize: First there is the *cost* of using this tool in the form of an associated learning curve. Second there is an intrinsic *risk* of a bad outcome arising from over-reliance or other forms of misuse.

The purpose of this document is to illustrate value, cost and risks of Agentic AI use in the context of research. *Research* usually means *data* and that in turn means *analysis* which means *methods* and often *coding* to implement those methods. *Coding* in turn implies *some*

programming language and *supporting compute infrastructure*. So there are a large number of gears in motion. My sub-thesis is that even if we are *not* writing code, the research context informs learning to use AI effectively.

> Assertion: Effective AI use has two components. First: Doing necessary tasks in less time. Second: Doing things that we previously ruled out or didn't consider or didn't know were possible. This is the expansion component. This writeup will attempt to describe AI skill development while trying to minimize hyperbole.

Rule 1: The 10-1-5 Efficiency Adjustment

Using AI incurs an **accountability deficit** that reduces perceived efficiency from an apparent factor of 10 to a factor of 2. The 10-1-5 rule states: This task will take me 10 hours. But wait! The Agentic AI has completed it in 1 hour, a factor of 10 improvement in efficiency. But this miracle has also incurred an accountability deficit that will take me an additional 4 hours to resolve. My efficiency went from 10x to 2x. Still acceptable.

An accountability deficit means: If I use AI-produced results without understanding them and they prove to be *wrong* and someone holds me accountable, then I could be scolded or subjected to derisive laughter or fired. For a researcher, even worse: I might have to retract my paper.

> Assertion: **Accountability deficit** can accumulate rapidly and often imperceptibly. Without the discipline to follow up it can remain unaddressed, increasing risk.

Rule 2: Stupidity freedom

> Assertion: The most important use of AI is as a learning tool.

Using Agentic AI to learn requires a shift of one's circa 2002 Google search behavior to a new mode of extended conversation. When we converse with another human we ascribe intelligence to that person. Conversely anything exhibiting *intelligent behavior* is easy to regard as human, thus the conversation (between humans) becomes subject to the rules of social interaction. One such rule is 'Do not ask the same question five times in a row.' But with a little practice this polite habit changes in working with Agentic AI.

If I am asking a car mechanic about the design and function of a cam shaft, I will monitor how many stupid questions I ask. When I get

up to about ten questions I might become award that I am starting to try the mechanic's patience. In contrast, with an Agentic AI I can be as intensely, deeply stupid as I need to be for as long as it takes comprehension to dawn. There is no second person to get frustrated, bored, annoyed or angry with me.

Conclude Part 1 by reconnecting the case studies with what we have so far in terms of efficiency and learning.

Part 2: Digging in

Part 1 introduced Agentic AI use as a skill. Here we will articulate strategies and examples.

An Agentic AI Framework

Building a working picture of how an Agentic AI operates requires some jargon. Without it we'll have a harder time keeping up with new developments — and AI is nothing but new developments. Here is a framework built on four components plus one critical constraint.

1. The Model

*Usually a Large Language Model (LLM). The Model receives a bundle of text — your prompt, the conversation history, system instructions, and other context — collectively the **payload**. It processes this payload and produces a response. That response goes to the Harness (see below), not directly to your screen. The Model might be telling the Harness to do something (run a command, read a file) rather than passing a message along to you. This is what makes the system "agentic" rather than merely conversational: The Model can request actions, not just generate text.*

*A crucial detail: The Model is **stateless**. It retains nothing between exchanges. Every time the Harness sends a new payload, it must include the entire conversation history — the Model has no memory of its own. This is why long conversations eventually hit a size limit called the **context window**. When the payload exceeds the context window, the system must either compress the history or start fresh. (This connects back to Rule 3.10: knowing when to start a new session.)*

2. The Harness

The orchestrating program that sits between you and the Model. In our case, this is the `kiro` IDE running on your laptop. The Harness is responsible for:

- *Assembling the payload (your prompt + conversation history + system instructions + any injected context)*
- *Sending the payload to the Model via an authenticated connection (which typically involves API keys and billing — someone is paying per token for each exchange)*
- *Receiving the Model's response and making a **decision**: Did the Model request an action (use a tool, read a file)? If so, execute it and send the result back for another turn. Did the Model produce a final message for the human? If so, display it and stop.*

This decision loop — "add another turn or halt" — is what distinguishes an Agentic AI from a simple chatbot. The Harness may execute multiple tool calls and multiple Model turns before anything appears on your screen. You type one prompt; behind the scenes, the system may cycle through several rounds of action and reasoning before presenting a result.

The authentication and billing machinery lives here too. Your Harness connects to a remote Model (possibly running in a data center far away) through an API layer that verifies your identity and meters usage. This is how providers charge for the service.

3. Tools

Tools are discrete capabilities the Harness can execute on the Model's behalf. Read a file. Run a shell command. Search the web. Write code to disk. Query a database. The Model doesn't do these things itself — it generates a structured request ("please read file X"), the Harness performs the action, and the result gets folded into the next payload.

Tools are what make the AI "agentic." Without them, the Model can only talk. With them, it can act on your project — editing files, running tests, fetching documentation, and reporting results back to you. In `kiro`, the built-in tools include file reading and writing, terminal command execution, web search, and code analysis.

4. Context & Steering

Everything that shapes the Model's behavior beyond its base training. This includes:

- *The conversation history (what you and the AI have said so far)*
- *System prompts (hidden instructions that define the AI's role and constraints)*
- ***Steering files**: persistent, reusable instructions you write and store in your project (in `.kiro/steering/`). These apply automatically to relevant conversations — for example, "always use pandas, not polars" or "follow our team's variable naming conven-*

tion." Steering is how you onboard the AI to your project once rather than repeating yourself every session.
- *Injected file contents (when you reference a file in your prompt, its contents become part of the payload)*

Context is the scarce resource in this system. The context window is finite, and everything competes for space within it: your prompt, the history, the steering rules, the contents of referenced files, and the results of tool calls. Managing what goes in — and recognizing when you've outgrown the window — is a practical skill (see Rules 3.5 and 3.10).

5. Skills

Skills are higher-level, reusable capabilities built on top of tools and steering. Where a tool is a single action ("read this file"), a skill is a coordinated workflow ("analyze this codebase for security vulnerabilities, then produce a summary report"). In `kiro`, skills can be defined and shared so that common multi-step processes don't need to be re-prompted from scratch each time.

Think of it this way: Tools are verbs. Steering is grammar. Skills are recipes.

A note on what this framework omits: There is a protocol called MCP (Model Context Protocol) that standardizes how the Harness discovers and connects to external tool providers — think of it as a "USB standard for AI tools." It matters if you're building or integrating custom tools, but for the purpose of using an Agentic AI as a research productivity tool, you can treat it as plumbing that operates beneath the surface. If you later find yourself wanting to connect the AI to a specialized database or departmental service, MCP is the mechanism that makes that possible without custom engineering.

Rule 3: Our most important *tactical* AI skill is *prompt writing*

- 3.1 Do not try to single shot complex prompts
- 3.2 Avoid the temptation to prescribe *how* the response is constructed; focus on *what you want to accomplish*
- 3.3 Avoid ambiguity; work with concrete cases or examples whenever possible
- 3.4 Refer back to and use language from previous prompt-response cycles
- 3.5 Avoid rapidity; consider whether some reflection on your part will advance the conversation more effectively
- 3.6 State your constraints and context up front. The AI doesn't know your deadline, your dataset size, your team's skill level, or your institutional policies unless you say so. A sentence of

context at the top of a prompt eliminates entire categories of irrelevant advice.

- 3.7 Ask the AI to push back. Prompts like "challenge my assumptions" or "what am I not considering?" turn the conversation from a vending machine into a sounding board. This is especially valuable when scoping a research question.
- 3.8 When a response is wrong or off-target, diagnose the prompt rather than just re-asking. Did you leave something ambiguous? Did you overload a single prompt with multiple goals? Prompt debugging is a learnable skill.
- 3.9 Use the conversation to build artifacts, not just get answers. Ask for outlines, checklists, draft emails, code scaffolds — things you can take away and refine. A prompt that produces a reusable artifact is worth more than one that produces a good paragraph of explanation.
  - Be on the lookout for when a response causes you to go "ahhhhh! I see it now!" This moment of insight should (often) be transcribed in project documentation.
- 3.10 Know when to start fresh. If a conversation has drifted far from your original intent, or the AI seems to be "stuck" in a pattern, a new session with a clearer opening prompt often outperforms continuing to course-correct.
- 3.11 As you realize good prompt-response dynamics: Be prepared to codify them as steering file instructions (persistent rules the AI follows across all future conversations in a project — see the Framework section above).

Rule 4: Develop a picture of Agentic AI Failure Modes

- *4.1* ***Hallucination****: The AI confidently states something that is factually wrong — a nonexistent function, a fabricated citation, a plausible but incorrect statistic.*
- *4.2* ***Drift****: Over a long conversation the AI gradually loses track of your original intent and begins solving a slightly different problem.*
- *4.3* ***Sycophancy****: The AI agrees with you rather than correcting you, reinforcing a mistaken assumption you brought into the conversation.*
- *4.4* ***Overreach****: The AI makes changes or decisions beyond what you asked — refactoring code you wanted left alone, adding features you didn't request.*
- *4.5* ***Stale context****: The AI relies on information from earlier in the conversation that you have since revised or retracted, producing internally inconsistent output.*
- *4.6* ***Shallow plausibility****: The output reads well and sounds authoritative but collapses under scrutiny — correct syntax wrap-*

ping incorrect logic, or a well-structured argument built on a false premise.

- *4.7* ***Tooling mismatch****: The AI suggests libraries, commands, or APIs that don't exist in your environment, version, or platform.*
- *4.8* ***Rat-holing****: The AI simply does not have the knowledge to do what you want; but it will gamely keep trying, often while making reassuring statements like "Oh I see the problem now…" In a memorable instance I was trying to get the AI to set up a computational task that would arrive at a known result; but try as it might the AI was unable to get there. Eventually and* ***without my noticing*** *it simply converted the code to print out the answer. This was an excellent example of* prompt exhaustion* leading to an invisible *accountability deficit* leading to *risk*: Had I not caught on (eventually) to this subterfuge, the Agentic AI 'solution' would have made me appear very foolish.

It may seem with all these pitfalls that perhaps using an Agentic AI is not such a great idea after all. This I would say is an open question; but certainly disaster and lost time can be mitigated by knowing and following successful practices.

To do: Rewrite rules 3 and 4 in non-AI tone

Rule 5: Step back to Meta-vate

We can step back from prompt-response cycles to take advantage of the simulation of human communication built into generative AI. The following are illustrative prompts.

Example: Generate Python code to produce a data chart showing daily temperature cycles and variability. Before you begin: Ask me at least three (but there is no upper limit here) clarifying questions to help insure we are on the same page.

Example: This topic is proving to be extremely complex. Let's evaluate my comprehension as follows: Make up a ten-question "exam". Each question is free response and they should get progressively more challenging. Present me with the exam and once I have answered all the questions let's review my responses and plan where to focus next.

Example: This project has an existing code base as well as a collection of supporting documents and publications. Examine this corpus through multiple lenses: Coding practice, consistency of code documentation, comprehensiveness of supporting documents, logical flow, readability, identifying redundancy, successful design in relation to stated objectives, other critical evaluations. Report on what

would improve matters; and also evaluate whether the goals of the program are clearly stated.

Example: I want to learn to be a responsible "open data / open code / open science" type of scientist. Tell me what that entails, what are best practices, what skills I will need (I understand that I should learn to use `git` for example); and help me develop a course of action that will realize this ambition. I particularly want to understand how to use testing (maybe 'unit tests'?) to reduce risk from dependence on code written by Agentic AI. I also want to know about what is involved in creating a Science Gateway so I can share my data with colleagues.

Example: Read this document and provide a critique, emphasizing gaps in the narrative flow, how the content can be improved, and what is likely to be confusing to the reader.

Rule 6: Working from specifications

The first rule for writing a computer program is *Close the laptop lid and take out a clean sheet of paper and a sharp pencil.*

Content

Rule 7: Test test test

Content

Part 3: Activities

Use an Agentic AI to…

- investigate / learn about / evaluate data security
  - Objective: Tied to the particular data system you choose
- explore a topic you understand in some depth
  - Objective: Evaluate the AI Agent's capacity to contribute
- locate, download and visualize some publicly available data
  - Investigate API and MCP servers
  - Objective: Evaluate the Agentic AI's capacity to transform a question ('What is this data telling us?') into a known quantity that could inform and advance our research agenda.
- learn how to create and make use of effective steering / guardrail content

To do: Add some additional guidance sub-bullets / topics to Activities.